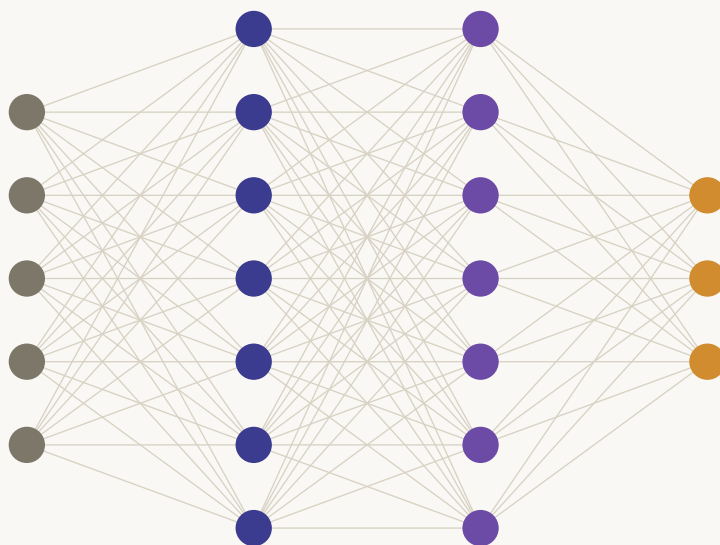


Learning From Examples

A practical, plain-language guide to deep learning — neurons, training, the four frameworks, and the Keras functions you will actually use.

From a single neuron to a convolutional network that reads handwriting at 98.7% accuracy — with a complete worked example, a framework comparison, and a Keras API reference.



Contents

Read straight through for a course, or jump to the reference sections at the back.

ORIENTATION	What this guide covers, and the data behind it	
PART I	What Deep Learning Is	
	Deep learning, machine learning, and AI	1.1
	When to use it — and when not to	1.2
PART II	The Neuron and the Network	
	What a single neuron does	2.1
	Activation functions	2.2
	Layers, depth, and what makes it “deep”	2.3
PART III	How a Network Learns	
	Loss functions: the feedback signal	3.1
	Gradient descent and backpropagation	3.2
	Epochs, batches, and the learning rate	3.3
	Optimisers	3.4
PART IV	The Frameworks	
	TensorFlow, PyTorch, JAX, and Keras	4.1
	The same model, written four ways	4.2
	Which one should you use?	4.3
PART V	Essential Keras Functions	
	Building models: Sequential, Functional, layers	5.1
	compile() — loss, optimiser, metrics	5.2
	fit(), evaluate(), predict()	5.3
	Callbacks, saving, and inspection	5.4
PART VI	Use Case: Reading Handwritten Digits	
	The complete end-to-end project	6.1
	Overfitting, and how we fixed it	6.2
	Going convolutional, and the final evaluation	6.3
APPENDIX	Full pipeline, glossary, and cheat sheet	

Orientation

What you are about to read, and the models behind it

This guide teaches deep learning from first principles — what a neuron computes, how a network learns, which framework to reach for, and which Keras functions you will use every day. It is written to be understood by someone meeting these ideas for the first time, but it does not stop at the surface.

Everything here was run. Every accuracy, curve, and failure in this guide came from training real models with **Keras 3.15 on TensorFlow 2.21**. Where a result is uncomfortable — and two of them genuinely are, including one where a classical model beats our neural network — the guide says so rather than smoothing it over.

The running example. We build a handwritten-digit classifier, the traditional first project in deep learning. Our dataset is the digits collection bundled with scikit-learn: **1,797 images** of handwritten numerals, each **8×8 pixels** in 16 grey levels, labelled 0-9. It is a smaller sibling of the famous MNIST dataset (which uses 28×28 images), chosen here because it needs no download and trains in seconds on a laptop CPU — so you can reproduce every number in this guide yourself.

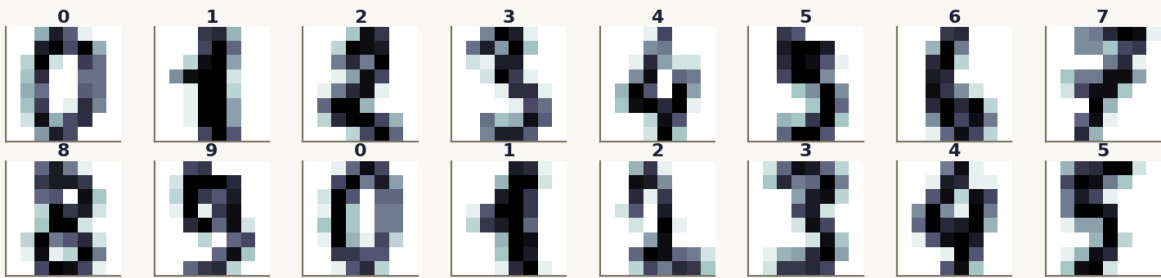


Figure 1 — Sixteen samples from the dataset with their labels. At 8×8 pixels these are crude, and some are genuinely ambiguous even to a human eye — which is exactly why the task is a useful test.

Property	Value	Why it matters here
Samples	1,797 digit images	1,347 train / 450 test throughout
Input	64 numbers (8×8 pixels, 0-16)	The network's input layer has 64 neurons
Output	10 classes (digits 0-9)	The output layer has 10 neurons + softmax
Best result here	98.67% (convolutional net)	6 errors out of 450

HOW TO READ Parts I-III build the concepts. **Part IV** compares TensorFlow, PyTorch, JAX and Keras and shows the same model written four ways. **Part V** is a reference for the Keras functions you will use daily — keep it open beside your editor. **Part VI** is the complete worked project. Boxes marked **THE CATCH** are the things that trip people up; read those even if you skim.

PART I

What Deep Learning Is

Where it sits, what it is good at, and — just as important — what it is not.

1.1 Deep learning, machine learning, and AI

Deep learning is machine learning using neural networks with many layers. Machine learning is any technique where a computer learns patterns from examples rather than being programmed with explicit rules. Artificial intelligence is the broadest term of the three — the general goal of getting machines to do things that look intelligent.

Deep learning is a subset of machine learning, which is one approach to AI

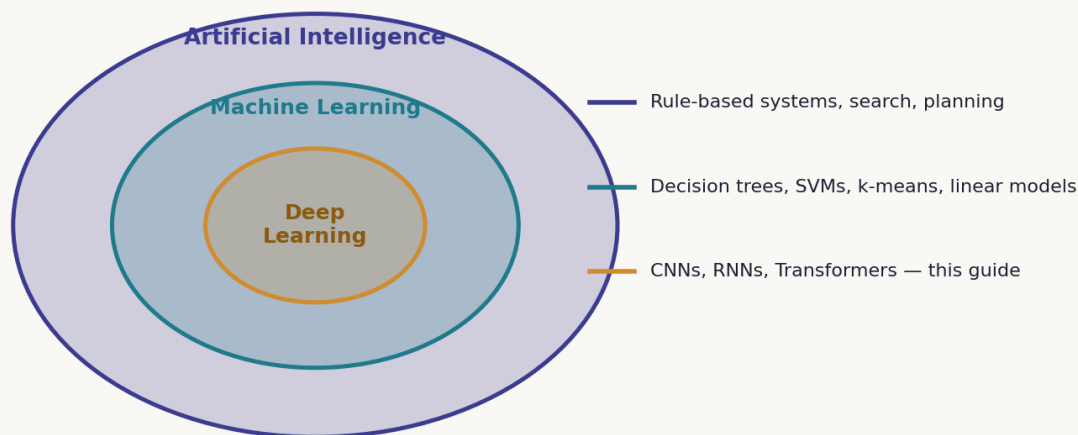


Figure 2 — The three terms are nested, not interchangeable. Every deep learning system is machine learning; not every machine learning system is deep learning.

An analogy. Traditional programming is writing a recipe: you specify every step, and the computer follows it. Machine learning is handing over a thousand finished dishes and their photographs, and letting the machine work out the recipe. Deep learning does that with a system deep enough to discover its own intermediate concepts along the way — first edges, then shapes, then whole objects — without anyone naming those concepts in advance.

That last point is the real distinction:

	Classical machine learning	Deep learning
Features	You engineer them by hand	The network learns them
Data needed	Often works with hundreds of rows	Usually thousands or more
Compute	Runs on a laptop	Often wants a GPU
Interpretability	Often readable (see a decision tree)	Largely opaque
Best at	Tabular data, small datasets	Images, audio, text, sequences

WHY IT TOOK SO LONG Neural networks are not new — the perceptron dates to the 1950s and backpropagation to the 1980s. What changed around 2010 was not the mathematics but the **circumstances**: enough labelled data (the internet), enough compute (GPUs, originally built for video games), and better training techniques. The ideas were waiting decades for the hardware to catch up.

1.2 When to use deep learning — and when not to

Deep learning is not the answer to every problem, and reaching for it by default is one of the most common mistakes a new practitioner makes. Here is an honest breakdown.

Good fits

Problem type	Example
Image recognition and segmentation	Detecting tumours in scans; classifying photographs
Natural language	Translation, summarisation, chat models
Speech and audio	Transcription, speaker identification
Sequence prediction	Forecasting from sensor streams
Generative tasks	Producing images, text, or audio resembling the training data

Poor fits — do not use deep learning for these

Situation	Why it fails
Very little training data	The network has millions of parameters and nothing to constrain them
You must explain every decision	Weights are not an explanation; use a tree or linear model
Inputs unlike anything in training	A model trained on cats and dogs will confidently mislabel a person
The rules are already known	If you have the equation, use the equation
Simple arithmetic or logic	Computing an average with a neural network is theatre

THE CATCH — AND WE PROVE IT LATER On our own digit dataset, a **support vector machine** — a classical, non-deep method from the 1990s — scores **99.11%**, beating our carefully built dense neural network at 97.56%. Deep learning only pulls ahead once we use a convolutional network that exploits the image structure (98.67%), and even then the SVM is still ahead. On small, clean datasets, classical methods are frequently better *and* cheaper. Always train a simple baseline first — see Part VI.

PART II

The Neuron and the Network

The single computation everything is built from, and how stacking it creates power.

2.1 What a single neuron does

A neuron performs exactly three operations: it **multiplies** each input by a weight, **adds** them up together with a bias, and passes the result through an **activation function**. That is the whole thing. Every network in this guide, and every large language model in the world, is built from this one operation repeated.

A single neuron: multiply, sum, add bias, activate

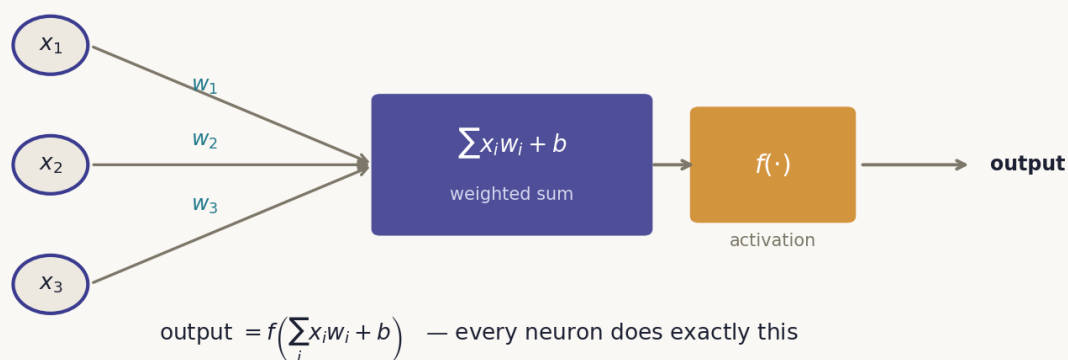


Figure 3 — The single computation at the heart of deep learning.

Written out:

```
output = f( x1w1 + x2w2 + x3w3 + ... + b )
```

x_i	the inputs	(data, or outputs from the previous layer)
w_i	the weights	(LEARNED — this is what training adjusts)
b	the bias	(LEARNED — shifts the threshold up or down)
f	the activation	(chosen by you, not learned)

A worked example. Take inputs $x = (0, 0.5, 1)$, weights $w = (-1, -0.5, 0.5)$, bias $b = 1$, and a ReLU activation. The weighted sum is $0 \times (-1) + 0.5 \times (-0.5) + 1 \times 0.5 = 0.25$. Add the bias: 1.25. Apply ReLU, which is $\max(0, x)$: the output is **1.25**. That is a complete forward pass through one neuron, and you can do it on paper.

WEIGHTS ARE THE KNOWLEDGE The weights and biases are the *only* thing that changes during training. A trained model is nothing but a large collection of these numbers — our little dense network holds **6,570** of them, and the convolutional one **151,306**. When you save a model to disk, those numbers are what you are saving.

2.2 Activation functions

The activation function is what makes a neural network more powerful than a linear model. Without one, stacking layers is pointless: a chain of matrix multiplications collapses algebraically into a single matrix multiplication, so a hundred layers would have exactly the expressive power of one.

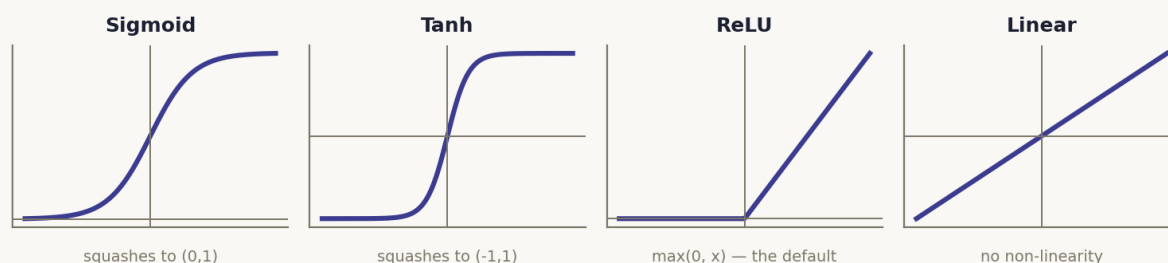


Figure 4 — The four activations you will meet most often.

Function	Formula	Range	Use it
ReLU	$\max(0, x)$	$[0, \infty)$	Default for hidden layers — fast, no vanishing gradient
Sigmoid	$1 / (1 + e^{-x})$	$(0, 1)$	Output layer for binary classification
Softmax	normalised exponentials	$(0,1)$, sums to 1	Output layer for multi-class — gives probabilities
Tanh	$\tanh(x)$	$(-1, 1)$	Sometimes in recurrent networks
Linear	x	$(-\infty, \infty)$	Output layer for regression

What we measured

We trained the identical 64→64→32→10 network four times, changing only the hidden-layer activation:

Hidden activation	Test accuracy
ReLU	0.9800
tanh	0.9711
none (purely linear)	0.9711
sigmoid	0.9600

ReLU wins, and sigmoid is the worst — the expected ordering, and the reason ReLU became the default. But notice something honest in that table: the purely linear network still reaches 97.11%. On a task this easy, a linear model is already nearly sufficient, so the activation buys less than you might expect. On harder problems the gap becomes enormous.

THE CATCH — WHY SIGMOID FELL OUT OF FAVOUR Look at the sigmoid curve in Figure 4: for large positive or negative inputs it flattens out, so its gradient approaches zero. In a deep network those near-zero gradients multiply together layer after layer until the signal reaching the early layers is effectively nothing — the **vanishing gradient problem**, which stalled deep networks for years. ReLU's gradient is exactly 1 for positive inputs, so it does not shrink. That single property is much of why deep learning started working.

2.3 Layers, depth, and what makes it “deep”

Neurons are organised into **layers**. Data enters at the input layer, passes through one or more **hidden layers**, and leaves at the output layer. “Deep” simply means there is more than one hidden layer — modern networks often have tens or hundreds.

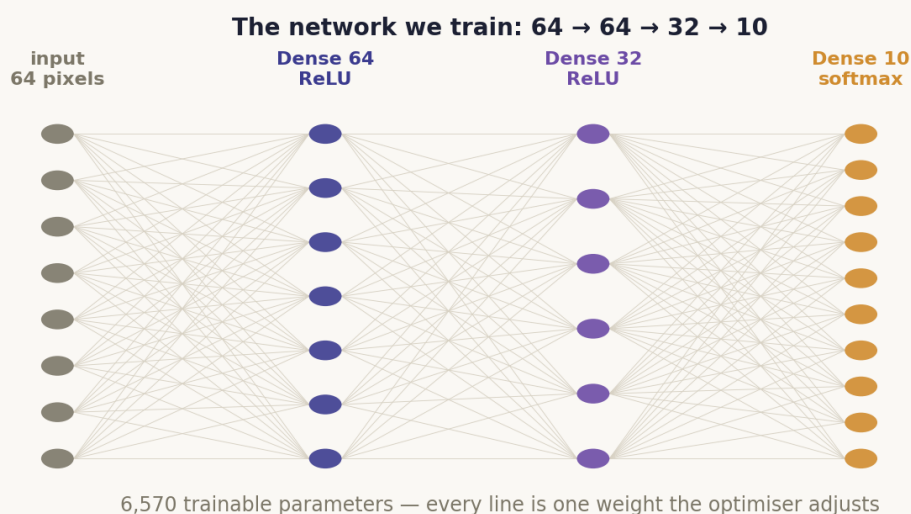


Figure 5 — The dense network we train in Part VI. Each of the 64 input neurons holds one pixel; each of the 10 output neurons holds the probability of one digit.

Why depth helps: the hierarchy of features

Each layer transforms the representation it receives into something slightly more abstract. In an image network, the first hidden layer typically learns to detect **edges**; the next combines edges into **corners and curves**; the next assembles those into **parts** (an eye, a wheel); and the last recognises whole **objects**. Nobody programs this hierarchy — it emerges because it is the most efficient way to reduce the loss.

This is the answer to “why not just use one enormous layer?” A single wide layer would have to map raw pixels directly to digits in one step. Depth lets the network solve the problem in stages, reusing simple discoveries to build complex ones.

THE CATCH More layers is not automatically better. Deeper networks are harder to train (vanishing gradients), need more data, and overfit more readily. Our 563,722-parameter network in Part VI does *worse* per unit of effort than the 6,570-parameter one, and reaches its best validation loss after just **7 epochs** before degrading. Match the model's capacity to the size of your data.

The layer types you will meet

Layer	What it is for	Keras
Dense	Every neuron connects to every input — general purpose	<code>layers.Dense(64)</code>
Conv2D	Slides small filters over an image to detect local patterns	<code>layers.Conv2D(32, 3)</code>
MaxPooling2D	Shrinks an image by keeping the strongest signal in each patch	<code>layers.MaxPooling2D(2)</code>
Flatten	Turns a multi-dimensional tensor into a flat vector	<code>layers.Flatten()</code>
Dropout	Randomly silences neurons during training to fight overfitting	<code>layers.Dropout(0.5)</code>
LSTM / GRU	Processes sequences, keeping a memory of what came before	<code>layers.LSTM(64)</code>
BatchNormalization	Rescales activations to keep training stable	<code>layers.BatchNormalization()</code>

PART III

How a Network Learns

Loss, gradients, and the loop that turns random weights into a working model.

3.1 Loss functions: the feedback signal

A **loss function** measures how wrong the model currently is, as a single number. Training is nothing more than the search for weights that make this number small.

An analogy. The loss is the score in a game the network is trying to lose. Every adjustment it makes is aimed at driving that score down. This means your choice of loss *defines* what the model considers success — pick the wrong one and the network will cheerfully optimise the wrong thing.

Choosing the right one

Your problem	Loss function	Keras name
Regression (predict a number)	Mean squared error	"mean_squared_error"
Regression, outliers present	Huber (grows linearly, not quadratically)	"huber"
Binary classification	Binary crossentropy	"binary_crossentropy"
Multi-class, integer labels	Sparse categorical crossentropy	"sparse_categorical_crossentropy"
Multi-class, one-hot labels	Categorical crossentropy	"categorical_crossentropy"

We used `sparse_categorical_crossentropy`, because our digits are ten classes labelled with plain integers 0–9. Had we one-hot encoded them into vectors, we would have used the non-sparse version. This is one of the most common beginner errors in Keras — the two losses differ only in the label format they expect.

A worked example: mean squared error

Prediction	Expected	Squared error
1	−1	4
2	−1	9
0	0	0
3	2	1
MSE		3.5

LOSS IS NOT A METRIC The **loss** is what the optimiser minimises — it must be smooth and differentiable. A **metric** is what you read to judge the model, and it need not be either. Accuracy is a metric, not a loss: it changes in jumps as predictions flip from wrong to right, so it gives the optimiser nothing to follow. That is why `compile()` takes both.

3.2 Gradient descent and backpropagation

Gradient descent is how the network improves: compute which direction each weight should move to reduce the loss, then take a small step that way. **Backpropagation** is the algorithm that computes those directions efficiently.

An analogy. You are standing on a foggy hillside and want to reach the valley floor. You cannot see the bottom, but you can feel which way the ground slopes beneath your feet. So you take a step downhill, feel again, and step again. The slope is the **gradient**; the size of your step is the **learning rate**. Repeat enough times and you arrive at a low point.

The training loop, in five steps

Step	What happens	Name
1	Run a batch of data through the network to get predictions	Forward pass
2	Compare predictions to the true labels	Loss computation
3	Work backwards, computing how much each weight contributed to the error	Backpropagation
4	Nudge every weight a small step in the direction that lowers the loss	Optimiser step
5	Repeat with the next batch, and the next, for many passes over the data	Training

Backpropagation is just the chain rule from calculus, applied cleverly. To know how a weight in the first layer affects the final loss, you multiply together the sensitivities along the path from that weight to the output. Doing this naively for millions of weights would be impossibly slow; backpropagation computes them all in a single backward sweep, reusing intermediate results.

YOU WILL NEVER WRITE THIS YOURSELF Every modern framework provides **automatic differentiation** — it watches the operations you perform and works out the gradients for you. This is one of the three things (with GPU execution and distribution) that every framework in Part IV exists to provide. In TensorFlow it is `GradientTape`; in PyTorch, `.backward()`; in JAX, `jax.grad()`. In Keras, `fit()` hides it entirely.

3.3 Epochs, batches, and the learning rate

Three numbers control the training loop, and getting them wrong is the most common reason a network fails to learn.

Term	Meaning	Ours
Batch size	How many samples are used to compute one weight update	32
Epoch	One complete pass over the whole training set	60
Learning rate	How big a step to take in the downhill direction	0.001 (Adam default)
Iteration / step	One weight update. $1,347 \text{ samples} \div 32 = 43 \text{ steps per epoch}$	$43 \times 60 = 2,580$

Why mini-batches?

You could compute the gradient using the entire training set at once (batch gradient descent) — it would be accurate but slow, and you would get only one weight update per epoch. Or you could use one sample at a time — fast updates, but each gradient is a noisy guess. **Mini-batch stochastic gradient descent** takes the middle path, and it is what essentially everyone uses. It is like polling: you do not need to ask every voter to get a usable estimate.

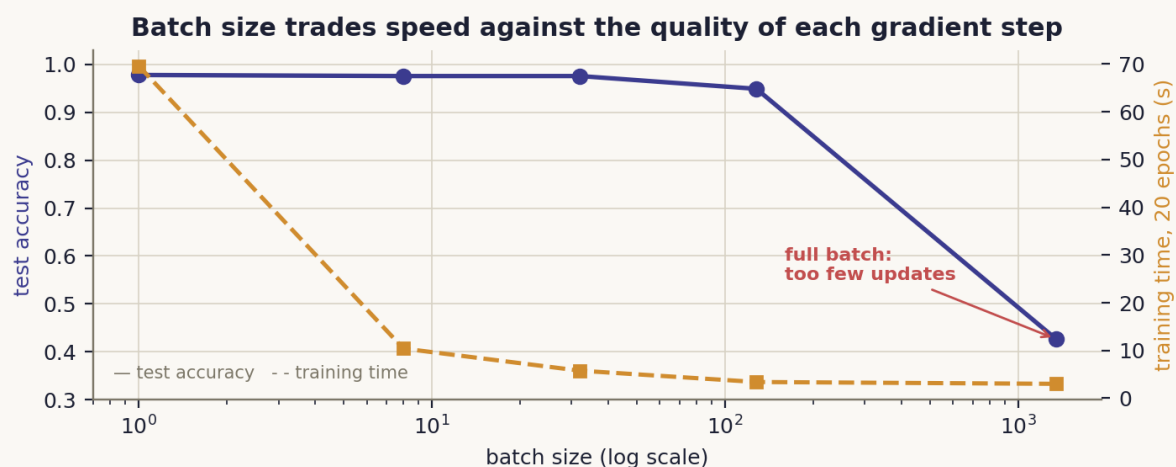


Figure 6 — Batch size on our data, 20 epochs each. Small batches give many noisy updates and cost time; huge batches are fast per epoch but starve the model of updates. Full-batch training (1,347) manages only 20 updates in total and collapses to 42.7%.

The learning rate is the one that will bite you

Set it too low and the model crawls, or never escapes its starting point. Set it too high and the steps overshoot the valley, bouncing up the far side — the loss rises instead of falling, or freezes. We swept it across five orders of magnitude with plain SGD:

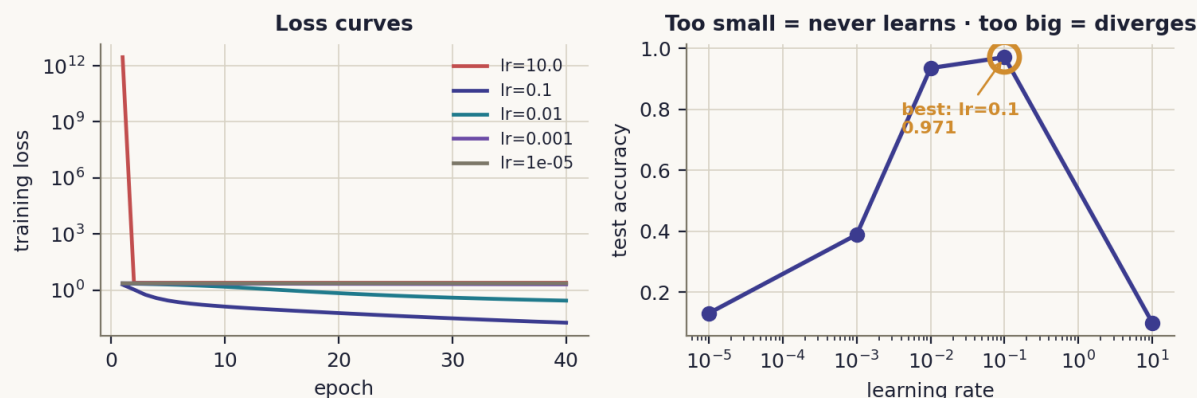


Figure 7 — Left: training loss for each learning rate. Right: the resulting test accuracy, forming the characteristic hump. Both extremes fail completely.

Learning rate	Test accuracy	What happened
10.0	0.1000	Diverged — steps overshoot; no better than guessing
0.1	0.9711	Just right
0.01	0.9356	Learning, but not finished in 40 epochs
0.001	0.3889	Far too slow to converge
0.00001	0.1311	Effectively frozen at its random initialisation

THE CATCH — DIAGNOSING A DEAD NETWORK A learning rate of 10.0 and one of 0.00001 both produce roughly 10% accuracy — random guessing on ten classes — but for opposite reasons. Look at the **loss curve**, not the accuracy: if the loss is wild or stuck high from the very first epoch, your rate is too big; if it decreases smoothly but imperceptibly, it is too small. This single diagnostic will save you hours.

3.4 Optimisers

An **optimiser** is the rule that turns gradients into weight updates. Plain gradient descent always steps a fixed fraction of the gradient; modern optimisers are cleverer — they build up momentum and adapt the step size per weight.

Optimiser	The idea	When to use
SGD	Step directly along the gradient	Simple, well understood; needs tuning
SGD + momentum	Accumulate velocity, like a ball rolling downhill	Rolls through small bumps and flat spots
RMSprop	Divide the step by a running average of recent gradient size	Good for recurrent networks
Adam	Momentum <i>and</i> per-weight adaptive step sizes	The default. Start here.

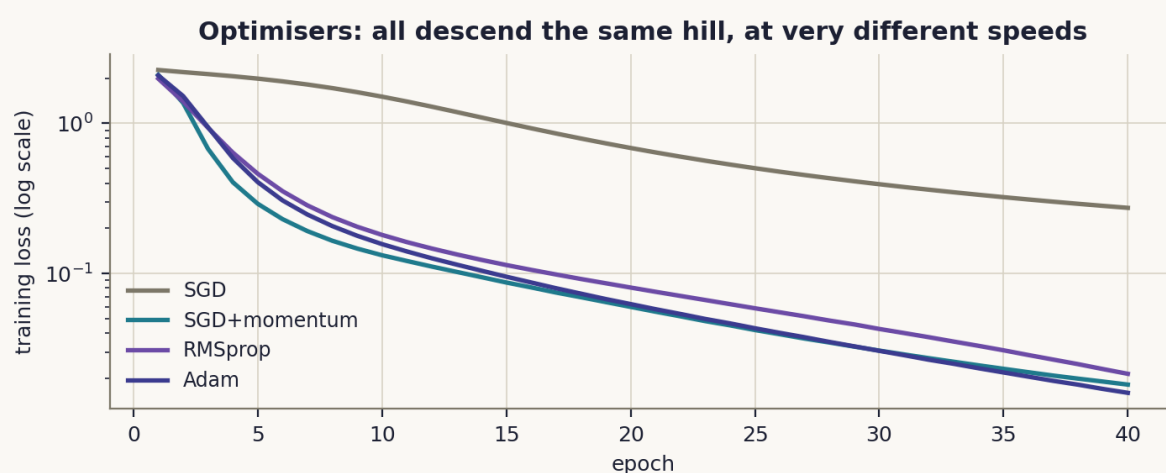


Figure 8 — The same network, the same data, four optimisers. Adam drives the training loss down fastest and furthest; plain SGD at the same learning rate is still an order of magnitude behind after 40 epochs.

Optimiser	Final training loss	Test accuracy
SGD (lr=0.01)	0.2736	0.9356
SGD + momentum 0.9	0.0182	0.9733
RMSprop	0.0214	0.9689
Adam	0.0160	0.9800

Adding momentum to SGD improves accuracy from 93.6% to 97.3% — a bigger jump than any architectural change we made. Adam, which combines momentum with per-weight adaptive rates, is best overall. This is why `optimizer="adam"` is the sensible default in almost every tutorial you will read.

PRACTICAL ADVICE Start with Adam at its default learning rate of 0.001 and change nothing else until the model works. Only then tune. If training is unstable, lower the rate by a factor of ten; if it is glacial, raise it. Well-tuned SGD with momentum sometimes generalises slightly better than Adam on large vision problems, which is why you still see it in research papers — but that is an optimisation to make last, not first.

PART IV The Frameworks

TensorFlow, PyTorch, JAX and Keras — what each is for, and how they fit together.

4.1 TensorFlow, PyTorch, JAX, and Keras

These four names dominate deep learning, and newcomers routinely assume they are four competing products to choose between. They are not. **Three of them are low-level engines; the fourth is a high-level interface that runs on top of any of them.**

An analogy. TensorFlow, PyTorch and JAX are raw building materials — timber, steel, concrete. Keras is a prefabricated building kit: it gives you walls and staircases ready-made, but it still has to be made of something. You pick the material underneath.

Keras 3 sits on top — write once, run on any of the three

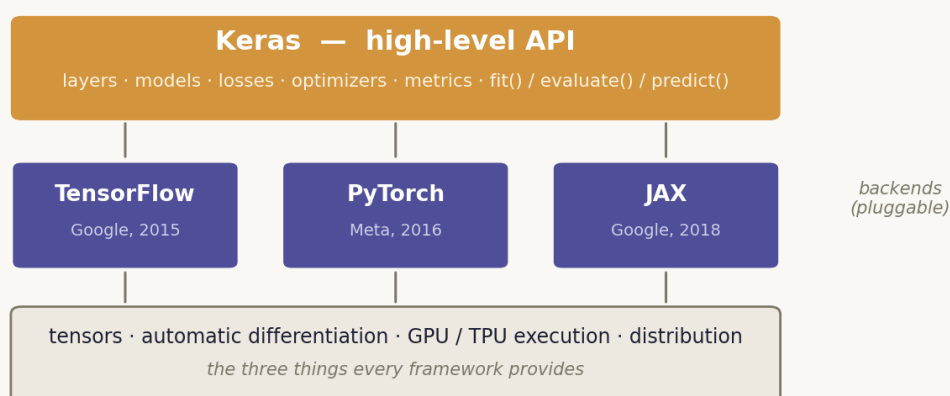


Figure 9 — How the four relate. Keras 3 is backend-agnostic: the same model code runs on TensorFlow, PyTorch, or JAX, and you switch with one environment variable.

What every framework provides

Underneath the differences, all three low-level frameworks exist to supply the same three capabilities. Together these unlock all of modern deep learning:

Capability	What it means
Automatic differentiation	Compute gradients of any differentiable expression, so you never hand-derive backpropagation
Accelerated tensor computation	Run array maths on GPUs and TPUs, not just the CPU
Distribution	Spread the work across many devices or many machines

A short history

Year	Event
2015 (March)	Keras launches as a friendly layer over Theano — the oldest of the four
2015 (Nov)	Google releases TensorFlow ; Keras adds support almost immediately
2016	Meta releases PyTorch , prized for eager execution and easy debugging
2018	Google releases JAX , built around functional purity and the XLA compiler

Year	Event
2023	Keras 3 arrives with pluggable backends: TensorFlow, PyTorch, <i>or</i> JAX
WHAT WE ACTUALLY RAN Every model in this guide was trained with Keras 3.15.1 on the TensorFlow 2.21 backend. The Keras code in Part V and Part VI would run unchanged on the PyTorch or JAX backends — that portability is the central design promise of Keras 3, and the reason it is a safe thing to learn first.	

4.2 The same model, written four ways

Nothing clarifies the relationship like seeing one task expressed in each. Below is the same idea — define a linear model, compute a loss, get gradients, update weights — in all four.

TensorFlow — explicit gradient tape

```
import tensorflow as tf

W = tf.Variable(tf.random.uniform((input_dim, output_dim)))
b = tf.Variable(tf.zeros((output_dim,)))

def model(x): return tf.matmul(x, W) + b

@tf.function
def training_step(x, y):
    with tf.GradientTape() as tape:
        loss = tf.reduce_mean(tf.square(y - model(x)))
    dW, db = tape.gradient(loss, [W, b])
    W.assign_sub(dW * lr); b.assign_sub(db * lr)
    return loss
```

PyTorch — gradients attached to the tensors

```
import torch

W = torch.rand(input_dim, output_dim, requires_grad=True)
b = torch.zeros(output_dim, requires_grad=True)

def model(x): return torch.matmul(x, W) + b

def training_step(x, y):
    loss = torch.mean(torch.square(y - model(x)))
    loss.backward()
    with torch.no_grad():
        W -= W.grad * lr; b -= b.grad * lr
    W.grad = None; b.grad = None
    return loss
```

JAX — functional and stateless

```
import jax, jax.numpy as jnp

def model(x, W, b): return jnp.matmul(x, W) + b

def compute_loss(state, x, y):
    W, b = state
    return jnp.mean(jnp.square(y - model(x, W, b)))

grad_fn = jax.value_and_grad(compute_loss)

@jax.jit
def training_step(x, y, W, b):
    loss, (dW, db) = grad_fn((W, b), x, y)
    return loss, W - dW * lr, b - db * lr
```

Keras — none of the above

```
import keras

model = keras.Sequential([keras.layers.Dense(output_dim)])
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(x, y, epochs=10, batch_size=32)
```

READ THOSE FOUR BLOCKS AGAIN This is the entire argument for Keras. The three low-level versions each require you to manage gradients, state, and the update rule by hand — and each has its own trap (PyTorch's accumulating gradients, JAX's mandatory statelessness). The Keras version is three lines and has no traps. You drop to the lower level only when you need something Keras does not express.

4.3 Which one should you use?

	TensorFlow	PyTorch	JAX
Made by	Google (2015)	Meta (2016)	Google (2018)
Style	Graph-oriented, compiles	Eager, imperative	Functional, stateless
Gradients	GradientTape	.backward()	jax.grad()
Speed	Fast	Slowest of the three	Usually fastest
Debugging	Harder once compiled	Easiest	Hardest
Deployment	Most mature	Good	Less developed
Research share	Declining	Dominant	Rising fast
Pretrained models	Less on Hugging Face	First-class on HF	Growing
Best at	Production pipelines	Research, prototyping	TPUs, very large scale

Honest recommendations

■ **If you are learning.** Start with **Keras**. You will build working models today rather than debugging gradient bookkeeping, and the concepts — layers, losses, optimisers, metrics — transfer directly to every other framework.

■ **If you are doing research or using pretrained models. PyTorch.** It dominates the research literature, and Hugging Face's model hub has first-class PyTorch support, so the model from last week's paper will be available in it.

■ **If you are shipping to production, mobile, or browsers. TensorFlow.** Its deployment ecosystem — TF-Serving, TFLite, TF.js — remains the most mature by a clear margin.

■ **If you need maximum speed or TPUs. JAX.** It is typically the fastest, and it was built for XLA and TPUs from the ground up. Expect more friction while writing the code.

■ **If you cannot decide.** Use **Keras 3** and defer the decision. Because the backend is pluggable, you can prototype on one, train on another, and deploy with a third — without rewriting your model.

Switching the Keras backend

the entire migration process

```
import os
os.environ['KERAS_BACKEND'] = 'jax'      # or 'tensorflow' or 'torch'
import keras                             # MUST come after the line above

print(keras.backend.backend())           # -> 'jax'
```

THE CATCH Note the string is 'torch', not 'pytorch' — the package is named torch. And the environment variable must be set *before* the first `import keras`; setting it afterwards silently does nothing, which is a genuinely confusing half-hour if you do not know.

PART V

Essential Keras Functions

A working reference to the API you will use every day. Keep this part open beside your editor.

5.1 Building models: Sequential, Functional, layers

There are three ways to define a model in Keras, in increasing order of flexibility.

1. Sequential — a straight stack of layers

The simplest and most common. Use it whenever your model is one input, one output, and a linear chain of layers — which covers most beginner projects.

```

model = keras.Sequential([
    keras.Input(shape=(64,)),          # declare input shape explicitly
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(10, activation='softmax'), # 10 classes -> softmax
])

```

2. Functional API — for anything branching

Needed when your model has multiple inputs or outputs, shared layers, or skip connections. You call each layer on a tensor and wire the graph explicitly.

```

inputs = keras.Input(shape=(64,))
x = keras.layers.Dense(64, activation='relu')(inputs)
x = keras.layers.Dense(32, activation='relu')(x)
outputs = keras.layers.Dense(10, activation='softmax')(x)
model = keras.Model(inputs=inputs, outputs=outputs)

```

3. Subclassing — full control

Write a Python class. Maximum flexibility, and the style PyTorch users will find familiar — at the cost of losing some of Keras's automatic checks.

```

class MyModel(keras.Model):
    def __init__(self):
        super().__init__()
        self.d1 = keras.layers.Dense(64, activation='relu')
        self.out = keras.layers.Dense(10, activation='softmax')

    def call(self, inputs):
        # the forward pass goes here
        return self.out(self.d1(inputs))

```

SHAPE INFERENCE Keras layers work out their own input shape the first time they see data, so you only ever declare the shape once, at the input. This is why `Dense(32)` needs no mention of what precedes it — the weights are created lazily, on the first call. If you want the weights to exist immediately (to inspect or load them), pass a `keras.Input` as we do above.

The most-used layer arguments

Argument	What it does	Example
units	Number of neurons in the layer	<code>Dense(64)</code>
activation	Activation function applied to the output	<code>activation='relu'</code>
input_shape / Input	Shape of one sample, excluding the batch dimension	<code>Input(shape=(64,))</code>
kernel_initializer	How the weights start out	<code>'he_normal'</code>
kernel_regularizer	Penalty on large weights (L1/L2)	<code>regularizers.l2(0.01)</code>

5.2 compile() — loss, optimiser, metrics

`compile()` configures *how* the model will learn. It does not train anything — it just attaches the three ingredients that `fit()` will use.

```
model.compile(
    optimizer='adam',           # HOW to update weights
    loss='sparse_categorical_crossentropy',  # WHAT to minimise
    metrics=['accuracy'],       # what to REPORT to you
)

# equivalently, with objects — needed to change any settings:
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss=keras.losses.SparseCategoricalCrossentropy(),
    metrics=[keras.metrics.SparseCategoricalAccuracy()],
)
```

Argument	Purpose	Common values
optimizer	The update rule (Part III.4)	'adam', 'sgd', 'rmsprop'
loss	The quantity minimised (Part III.1)	'sparse_categorical_crossentropy', 'mse'
metrics	Reported each epoch, not optimised	['accuracy'], ['mae'], [AUC()]

THE STRING SHORTCUTS ARE REAL OBJECTS Passing 'adam' is exactly equivalent to passing `keras.optimizers.Adam()` with all defaults. Use the string form while prototyping; switch to the object form the moment you need to change a setting — most often the learning rate, which you *cannot* set through the string.

5.3 fit(), evaluate(), predict()

These three do the actual work: train the model, score it, and use it.

fit() — run the training loop

```
history = model.fit(
    X_train, y_train,
    epochs=60,           # passes over the whole training set
    batch_size=32,       # samples per weight update
    validation_split=0.2, # hold back 20% to monitor generalisation
    callbacks=[...],     # optional hooks (see 5.4)
    verbose=0,           # 0 silent, 1 progress bar, 2 one line per epoch
)
```

It returns a History object. `history.history` is a plain dict mapping 'loss', 'val_loss', 'accuracy', 'val_accuracy' to one value per epoch — this is what you plot to produce learning curves like Figure 10.

evaluate() — score on held-out data

```
loss, acc = model.evaluate(X_test, y_test, verbose=0)
# -> returns the loss followed by each metric you compiled with
```

predict() — use the trained model

```
proba = model.predict(X_new, verbose=0) # shape (n_samples, 10)
labels = proba.argmax(axis=1)           # softmax -> the chosen class
confidence = proba.max(axis=1)          # how sure the model is
```

Method	Needs labels?	Returns	Use it to
fit()	Yes	History object	Train the model
evaluate()	Yes	Loss and metrics	Measure performance honestly
predict()	No	Raw output array	Make predictions on new data

THE CATCH — `validation_split` TAKES THE LAST ROWS `validation_split=0.2` slices off the *final* 20% of your arrays **before** shuffling. If your data is sorted by class — as many datasets are — your validation set will contain only the last few classes and your validation metrics will be nonsense. Shuffle first, or pass an explicit `validation_data=(X_val, y_val)`.

5.4 Callbacks, saving, and inspection

Callbacks are objects that hook into the training loop and act at the end of each epoch — the mechanism behind early stopping, checkpointing, and learning-rate schedules.

Callback	What it does
EarlyStopping	Stops training when a monitored metric stops improving
ModelCheckpoint	Saves the model (or just the best one) during training
ReduceLROnPlateau	Cuts the learning rate when progress stalls
TensorBoard	Logs everything for the TensorBoard visualiser
CSVLogger	Appends per-epoch metrics to a CSV file

early stopping + checkpointing

```
early = keras.callbacks.EarlyStopping(
    monitor='val_loss',      # what to watch
    patience=15,             # epochs of no improvement to tolerate
    restore_best_weights=True, # roll back to the best epoch – IMPORTANT
)
ckpt = keras.callbacks.ModelCheckpoint('best.keras', save_best_only=True)

model.fit(X, y, epochs=200, validation_split=0.2, callbacks=[early, ckpt])
```

Saving and loading

```
model.save('digits_cnn.keras')          # architecture + weights + optimiser
restored = keras.saving.load_model('digits_cnn.keras')

model.save_weights('w.weights.h5')      # weights only
model.load_weights('w.weights.h5')
```

We verified this round trip: our saved convolutional model is **1,812 KB** on disk, and the reloaded copy scores **0.9867** on the test set — identical to the original, to the last decimal place.

THE ONE-LINE HABIT WORTH FORMING Call `keras.utils.set_random_seed(42)` at the top of every script. Neural networks start from random weights, so two runs of identical code give different results — which makes it impossible to tell whether your change helped or you simply got a luckier initialisation. Every number in this guide is reproducible because of that one line.

Inspecting a model

Call	What you get
<code>model.summary()</code>	Table of layers, output shapes, and parameter counts
<code>model.count_params()</code>	Total number of trainable numbers (6,570 for our dense net)
<code>model.layers</code>	The list of layer objects
<code>layer.get_weights()</code>	The actual weight arrays, as NumPy
<code>keras.utils.plot_model(model)</code>	A diagram of the architecture

Data preparation helpers

Call	What it does
<code>keras.utils.to_categorical(y)</code>	Integer labels → one-hot vectors
<code>keras.utils.set_random_seed(42)</code>	Make a run reproducible across Python, NumPy and the backend
<code>keras.utils.image_dataset_from_directory(p)</code>	Build a dataset from folders of images

PART VI

Use Case: Reading Handwritten Digits

The complete project, start to finish, with everything that went wrong along the way.

6.1 The complete end-to-end project

We now build the whole thing: take 1,797 images of handwritten digits and produce a model that reads them. This follows the standard deep learning workflow, and we report what actually happened at each step — including the step where a classical model embarrassed our neural network.

Steps 1-3. Frame the problem, identify inputs and outputs, prepare the data

The problem is **multi-class classification**: ten possible digits. Each input is an 8×8 image, which we flatten into 64 numbers; each output is one of ten classes. Pixel values run 0–16, so we divide by 16 to put everything in the range 0–1 — neural networks train far more reliably on small, similarly-scaled inputs. Then we split off 25% of the data as a test set the model will not see until the very end.

```

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)  # stratify keeps class balance
X_tr, X_te = X_tr / 16.0, X_te / 16.0                # scale to 0..1

```

Step 4. Baselines first — before any deep learning

This step is skipped constantly and it should never be. Before building a network, find out what simple methods achieve, so you know whether the network is worth its complexity.

Baseline	Test accuracy	Verdict
Always predict the most common class	0.1022	The floor — anything must beat this
Logistic regression	0.9622	Already excellent
Support vector machine	0.9911	Better than any network we will build

SIT WITH THAT RESULT An SVM with default settings scores 99.11%. Our neural networks will reach 97.56% and 98.67%. On a small, clean, low-resolution dataset, the classical method wins — and it trains in a fraction of the time. Deep learning's advantage appears with *scale*: more data, higher resolution, messier inputs. The honest lesson of this project is that you should always know your baseline.

Steps 5-6. Build, compile, train

```
import keras
from keras import layers
keras.utils.set_random_seed(42)

model = keras.Sequential([
    keras.Input(shape=(64,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(10, activation='softmax'),
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history = model.fit(X_tr, y_tr, epochs=60, batch_size=32,
                    validation_split=0.2, verbose=0)
```

Result: 97.56% test accuracy from 6,570 parameters, trained in 17 seconds on a CPU. That comfortably beats logistic regression, and loses to the SVM.

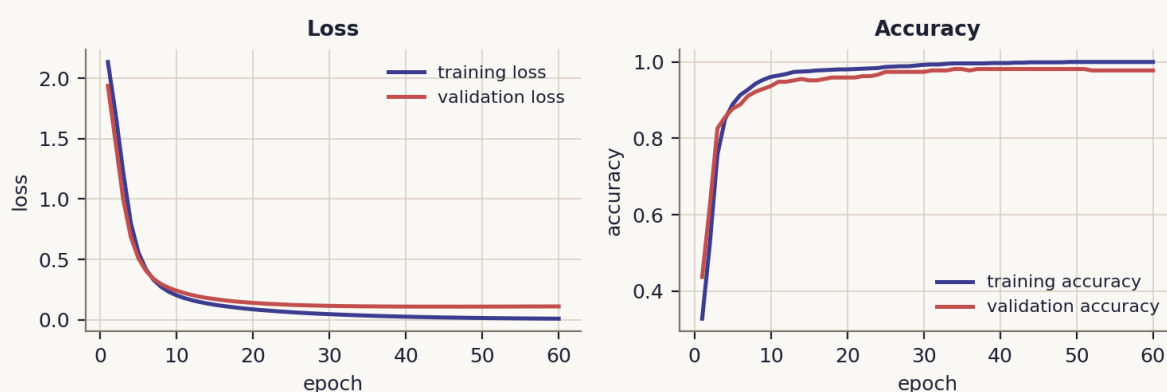


Figure 10 — The learning curves returned by `fit()`. Loss falls and accuracy rises; the validation curves track the training curves closely, which is what healthy training looks like.

6.2 Overfitting, and how we fixed it

To show the central failure mode of deep learning, we deliberately built a network far too large for this dataset: three hidden layers of 512 neurons, **563,722 parameters** — roughly 400 parameters for every training image.

It did exactly what theory predicts. Training loss fell to **0.0000** and training accuracy hit **100%** — the network had memorised all 1,347 training images perfectly. But validation loss reached its best value at **epoch 7** and got steadily worse for the remaining 93 epochs. The model was still ‘improving’ on paper while genuinely getting worse at its job.

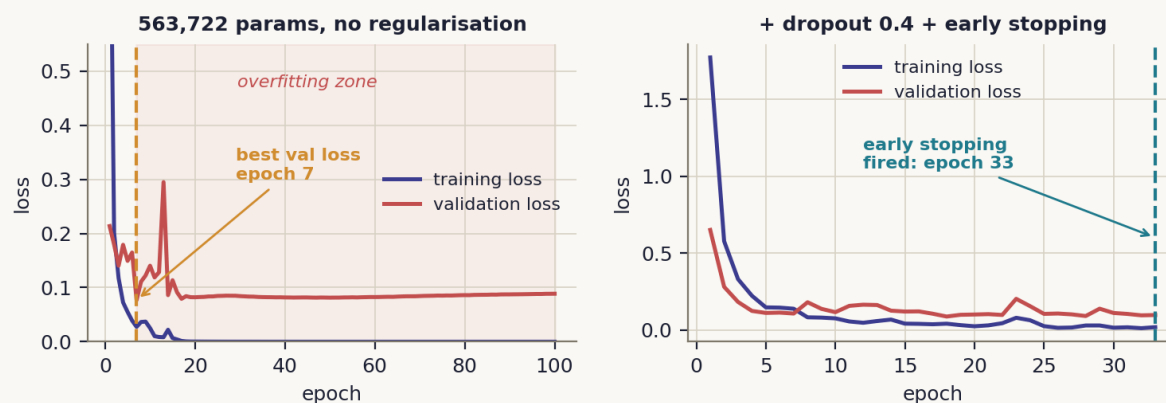


Figure 11 — Left: the unregularised network. After epoch 7 the two curves separate — training loss keeps falling while validation loss climbs. That gap is overfitting, and it is the single most important pattern to recognise in deep learning. Right: with dropout and early stopping, training halts automatically at epoch 33.

The standard remedies

Technique	What it does	Keras
Early stopping	Stop when validation loss stops improving, and roll back	<code>callbacks.EarlyStopping</code>
Dropout	Randomly zero a fraction of neurons each step, so no neuron is indispensable	<code>layers.Dropout(0.4)</code>
Weight regularisation	Penalise large weights, favouring simpler functions	<code>regularizers.l2(0.01)</code>
More data	The best fix of all, when available	– (or augmentation)

We applied dropout of 0.4 after each hidden layer plus early stopping with patience 15:

Model	Epochs run	Best val loss	Test accuracy
Oversized, unregularised	100	0.0769	0.9778
+ dropout 0.4 + early stopping	33	0.0864	0.9756

AN HONEST RESULT Regularisation did **not** improve accuracy here — 97.56% versus 97.78% is a wash, and dropout of 0.4 is arguably too aggressive for a dataset this small. What it *did* buy is a model trained in **a third of the epochs**, stopped automatically at the right moment, with no need to guess the epoch count in advance. On larger and noisier problems the accuracy benefit appears too. Reporting this as a triumph would have been dishonest; the technique is still correct.

6.3 Going convolutional, and the final evaluation

Our dense network throws away something important: it flattens the 8×8 image into 64 unrelated numbers, so it has no idea which pixels are neighbours. A **convolutional network** keeps the grid intact and slides small learned filters across it, detecting local patterns wherever they occur.

An analogy. The dense network is reading a photograph as a shuffled list of pixel values. The convolutional network is looking at the photograph. Since a stroke of ink is a stroke of ink wherever it appears, detecting it with one reusable filter is both more efficient and more robust than learning it separately for every position.

```
X_tr_img = X_tr.reshape(-1, 8, 8, 1)    # restore the grid: (samples, h, w, channels)

cnn = keras.Sequential([
    keras.Input(shape=(8, 8, 1)),
    layers.Conv2D(32, 3, activation='relu', padding='same'),
    layers.Conv2D(64, 3, activation='relu', padding='same'),
    layers.MaxPooling2D(2),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax'),
])
cnn.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
cnn.fit(X_tr_img, y_tr, epochs=60, batch_size=32, validation_split=0.2)
```

Result: 98.67% test accuracy — 6 errors out of 450. That is a 45% reduction in errors compared with the dense network, from respecting the structure of the data.

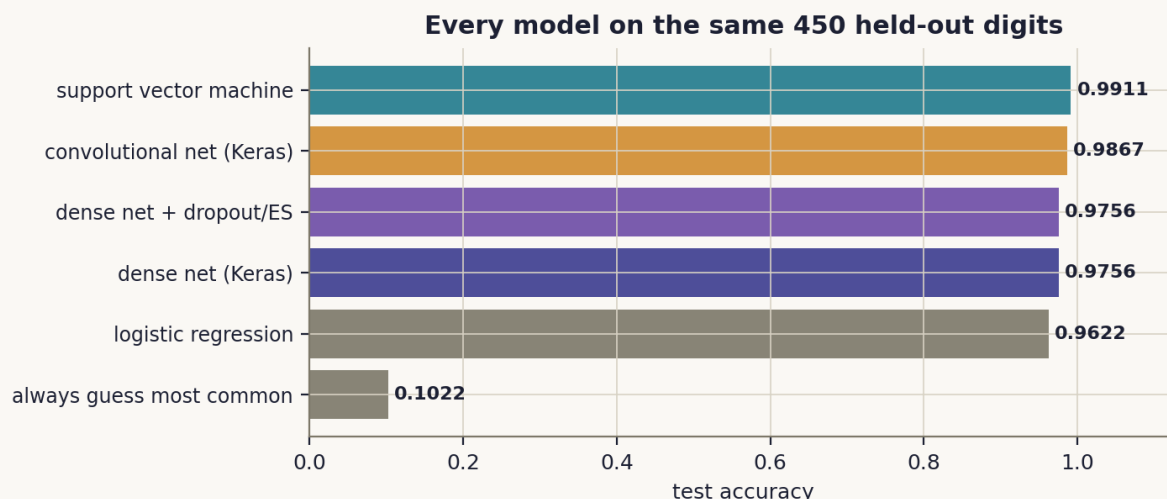


Figure 12 — Every model in this guide on the same 450 held-out digits.

The final evaluation

Where the CNN goes wrong — and the digits it got wrong

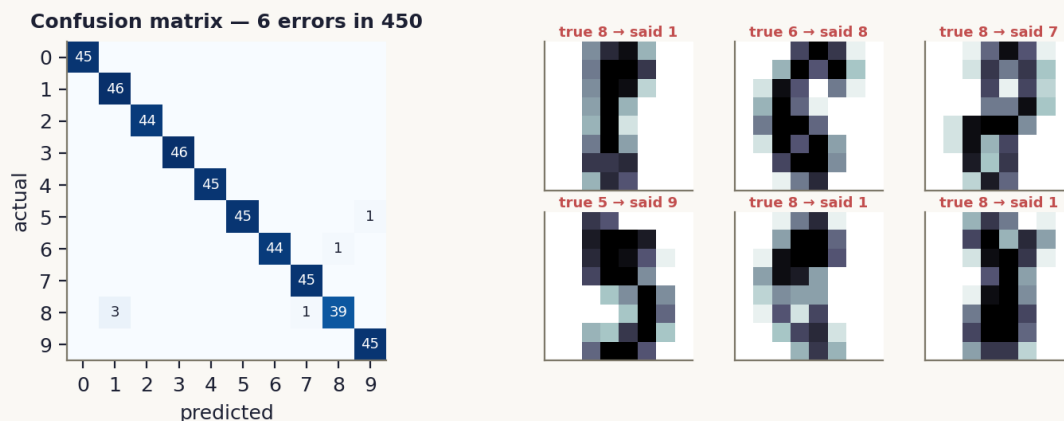


Figure 13 — The confusion matrix, and every digit the model got wrong. Four of the six errors involve the digit 8 — at 8×8 resolution it genuinely resembles 1 and 7.

Digit	Precision	Recall	Note
0, 2, 3, 4	1.0000	1.0000	Perfect
1	0.9388	1.0000	Caught all, but three 8s were misfiled as 1
8	0.9750	0.9070	The hardest digit — 4 of the 6 errors
Overall	0.9870	0.9867	444 of 450 correct

A USEFUL SIGNAL — THE MODEL KNOWS WHEN IT IS UNSURE We checked the softmax confidence on correct versus incorrect predictions. When the model was right, its mean confidence was **0.998**. When it was wrong, **0.819**. The gap means confidence is a usable warning flag: in production you could route any prediction below a threshold to a human reviewer and catch most errors. Not all models are so well calibrated — but it is always worth checking.

PART A
Appendix

The complete pipeline, a glossary, and the one-page version.

A1 · The complete pipeline

Everything in this guide, start to finish. This is the code that produced every number quoted.

```

import os
os.environ['KERAS_BACKEND'] = 'tensorflow'      # or 'torch' or 'jax'
import keras
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix

keras.utils.set_random_seed(42)                # reproducibility, always

# ---- data ----
X, y = load_digits(return_X_y=True)            # 1797 x 64, 10 classes
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
X_tr, X_te = X_tr / 16.0, X_te / 16.0          # scale to 0..1

# ---- baseline FIRST (it wins on this dataset: 0.9911) ----
print('SVM:', SVC().fit(X_tr, y_tr).score(X_te, y_te))

# ---- the dense network: 0.9756 ----
model = keras.Sequential([
    keras.Input(shape=(64,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(10, activation='softmax'),
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.summary()
hist = model.fit(X_tr, y_tr, epochs=60, batch_size=32,
                 validation_split=0.2, verbose=0)
print('dense:', model.evaluate(X_te, y_te, verbose=0)[1])

# ---- the convolutional network: 0.9867 ----
X_tr_i, X_te_i = X_tr.reshape(-1,8,8,1), X_te.reshape(-1,8,8,1)
cnn = keras.Sequential([
    keras.Input(shape=(8,8,1)),
    layers.Conv2D(32, 3, activation='relu', padding='same'),
    layers.Conv2D(64, 3, activation='relu', padding='same'),
    layers.MaxPooling2D(2), layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(128, activation='relu'), layers.Dropout(0.5),
    layers.Dense(10, activation='softmax'),
])
cnn.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy', metrics=['accuracy'])
early = keras.callbacks.EarlyStopping(monitor='val_loss', patience=15,
                                     restore_best_weights=True)
cnn.fit(X_tr_i, y_tr, epochs=60, batch_size=32,
       validation_split=0.2, callbacks=[early], verbose=0)

# ---- evaluate properly ----
pred = cnn.predict(X_te_i, verbose=0).argmax(axis=1)
print(confusion_matrix(y_te, pred))
print(classification_report(y_te, pred, digits=4))

# ---- ship it ----
cnn.save('digits_cnn.keras')
restored = keras.saving.load_model('digits_cnn.keras')

```

A2 · Glossary

Term	Meaning
Neuron	Computes $f(\sum x_i w_i + b)$ — multiply, sum, activate
Weight / bias	The learned numbers. A trained model is nothing else
Activation function	The non-linearity applied to a neuron's output (ReLU, sigmoid, softmax)
Layer	A group of neurons applied together; the unit you assemble models from
Dense / fully connected	A layer where every neuron sees every input
Deep	Having more than one hidden layer
Forward pass	Running data through the network to get a prediction
Loss function	The single number measuring wrongness, which training minimises
Metric	A number reported for humans; need not be differentiable
Gradient descent	Repeatedly stepping weights in the direction that lowers the loss
Backpropagation	The efficient algorithm for computing all those gradients at once
Automatic differentiation	The framework feature that does backpropagation for you
Learning rate	Step size. The most important hyperparameter
Batch size	Samples used per weight update
Epoch	One full pass over the training data
Optimiser	The update rule: SGD, momentum, RMSprop, Adam
Overfitting	Memorising training data; train loss falls while validation loss rises
Dropout	Randomly disabling neurons in training to prevent co-dependence
Early stopping	Halting when validation stops improving, keeping the best weights
Softmax	Turns output scores into probabilities summing to 1
CNN	Convolutional network; slides learned filters over grid-shaped data
Tensor	A multi-dimensional array — the data structure everything is built on
Backend	The engine Keras runs on: TensorFlow, PyTorch, or JAX

A3 · The one-page version

If you remember nothing else...	Because
Always train a simple baseline first	An SVM beat both our networks: 0.9911 vs 0.9867
Scale your inputs	Networks train far more reliably on values near 0-1
Start with Adam at its defaults	0.9800 vs 0.9356 for untuned SGD
The learning rate matters most	Too high and too low both give ~10% — for opposite reasons
Watch validation loss, not training loss	Our big net hit 100% train accuracy while getting worse
Use EarlyStopping with <code>restore_best_weights</code>	Stopped at epoch 33 instead of 100, same accuracy
Match the architecture to the data	A CNN cut errors 45% by respecting the image grid
Set a random seed	Otherwise you cannot tell a real improvement from luck
Learn Keras first	Three lines vs a hand-written loop — and it runs on all three backends

A4 · The workflow, as a checklist

#	Step	Watch out for
1	Frame the problem — classification or regression?	Decides your loss and output layer
2	Identify inputs and outputs	Output neurons = number of classes
3	Prepare and split the data	Scale inputs; stratify the split; never touch the test set
4	Establish a baseline	If a simple model wins, stop here
5	Choose an architecture	Match it to the data shape; start small
6	Pick loss, optimiser, metrics	<code>sparse_</code> vs plain crossentropy depends on label format
7	Train, watching validation	The gap between the curves is the whole story
8	Evaluate on held-out data	Confusion matrix, not just accuracy
9	Refine	Tune the learning rate before anything else
10	Save and share the model	<code>model.save('name.keras')</code>

WHERE TO GO NEXT The natural next steps: **transfer learning** (start from a pretrained network instead of random weights — the single highest-value technique for small datasets); **data augmentation** to multiply your training set; **convolutional architectures** in depth (ResNet, EfficientNet); and **Transformers**, the architecture behind modern language models. The Keras guides at keras.io and Chollet's *Deep Learning with Python* are the natural companion reading; Michael Nielsen's free *Neural Networks and Deep Learning* is the best treatment of backpropagation from first principles.